

PATERNI PONAŠANJA

Strategy patern

Strategy patern koristi se kada postoji potreba za izdvajanjem različitih algoritama u zasebne klase. Pogodan je u situacijama kada za isti problem postoji više mogućih načina rješavanja, a klijent bira algoritam koji želi koristiti. Implementacija algoritama je potpuno odvojena od klijenta koji ih koristi, čime se postiže veća fleksibilnost sistema.

U nekim budućim nadogradnjama sistema frizerskog salona „Sakura“ ovaj patern može biti primijenjen prilikom slanja obavijesti korisnicima. Na primjer, korisnik može odabrati da obavještenja o rezervacijama, akcijama ili newsletter sadržajima prima putem e-maila ili SMS poruka. Svaki način slanja predstavljao bi zasebnu strategiju, dok bi sistem u zavisnosti od korisničkih preferencija odabirao odgovarajući algoritam za slanje obavijesti.

State patern

State patern omogućava objektu da mijenja svoje ponašanje u zavisnosti od trenutnog stanja. Posebno je koristan kada objekat može prelaziti između više stanja tokom svog životnog ciklusa, pri čemu svako stanje definiše drugačije ponašanje.

U sistemu „Sakura“ ovaj patern može biti iskorišten za upravljanje statusima rezervacija. Rezervacija može imati stanja poput „Potvrđena“, „Otkazana“ ili „Završena“. U zavisnosti od trenutnog stanja, sistem dozvoljava različite akcije. Na primjer, potvrđena rezervacija može biti otkazana ili realizovana, dok završena rezervacija više ne može mijenjati svoj status. Primjenom State paterna logika za svako stanje bila bi izdvojena u posebne klase, čime bi se izbjegla velika količina uslovnih grananja.

Template Method patern

Template Method patern koristi se kada postoji potreba za definisanjem zajedničke strukture algoritma, dok se pojedini koraci algoritma implementiraju na različite načine u podklasama. Na taj način zadržava se jedinstven tok izvršavanja, ali se omogućava prilagođavanje određenih dijelova procesa.

U sistemu „Sakura“ ovaj patern može biti primijenjen prilikom procesa registracije različitih tipova korisnika. Osnovni koraci registracije, poput unosa podataka, validacije i kreiranja naloga, bili bi zajednički za sve korisnike. Međutim, pojedini koraci mogli bi se razlikovati u zavisnosti od toga da li se registruje klijent, frizer ili administrator sistema.

Observer patern

Observer patern uspostavlja relaciju između objekata tako da promjena stanja jednog objekta automatski izaziva obavješćavanje svih povezanih objekata. Na taj način se postiže efikasna komunikacija između komponenti sistema bez njihove direktne povezanosti.

U sistemu frizerskog salona „Sakura“ ovaj patern prirodno se može primijeniti na funkcionalnost newslettera. Korisnici koji se prijave za primanje newsletter poruka predstavljaju posmatrače (Observer), dok newsletter sistem predstavlja subjekt (Subject). Kada administrator objavi novu promociju, svi prijavljeni korisnici automatski dobijaju odgovarajuću notifikaciju. Na isti način moguće je obavješćavati korisnike o novim uslugama ili proizvodima koji su dodani u ponudu salona.

Iterator patern

Iterator patern omogućava prolazak kroz kolekciju podataka bez potrebe za poznavanjem njene interne strukture. Različiti iteratori mogu koristiti različite načine prolaska kroz istu kolekciju, dok klijent koristi jedinstven interfejs za pristup podacima.

U sistemu „Sakura“ Iterator patern može se primijeniti za pregled proizvoda, usluga ili rezervacija. Na primjer, korisnik može pregledavati dostupne proizvode ili usluge bez potrebe da poznaje način na koji su ti podaci pohranjeni u sistemu.

Command patern

Command patern pretvara zahtjev u poseban objekat koji sadrži sve informacije potrebne za izvršavanje određene akcije. Na taj način omogućava se odgađanje izvršavanja zahtjeva, njihovo evidentiranje ili jednostavnije upravljanje različitim akcijama unutar sistema.

U sistemu frizerskog salona „Sakura“ ovaj patern mogao bi se koristiti za upravljanje rezervacijama termina. Akcije poput kreiranja rezervacije, otkazivanja rezervacije ili potvrđivanja termina mogle bi biti predstavljene kao zasebne komande. Time bi se omogućilo lakše evidentiranje aktivnosti korisnika, kao i eventualna implementacija funkcionalnosti poništavanja određenih akcija u budućim verzijama sistema.